

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Институт № 8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Курсовой проект по курсу
«Операционные системы»

Студент: Салихов Руслан Ринатович

Группа: М8О-203Б-23

Вариант: 17

Преподаватель: Миронов Евгений Сергеевич

Оценка: _____

Дата: _____

Подпись: _____

Москва, 2024

Постановка задачи

Исследование 2 аллокаторов памяти: необходимо реализовать два алгоритма аллокации памяти и сравнить их по следующим характеристикам:

- 1) Фактор использования
- 2) Скорость выделения блоков
- 3) Скорость освобождения блоков
- 4) Простота использования аллокатора

Каждый аллокатор памяти должен иметь функции аналогичные стандартным функциям `free` и `malloc` (`realloc`, опционально). Перед работой каждый аллокатор инициализируется свободными страницами памяти, выделенными стандартными средствами ядра. Необходимо самостоятельно разработать стратегию тестирования для определения ключевых характеристик аллокаторов памяти. При тестировании нужно свести к минимуму потери точности из-за накладных расходов при измерении ключевых характеристик, описанных выше. В отчете необходимо отобразить следующее:

- 1) Подробное описание каждого из исследуемых алгоритмов
- 2) Процесс тестирования
- 3) Обоснование подхода тестирования
- 4) Результаты тестирования
- 5) Заключение по проведенной работе

Вариант 17: Необходимо сравнить два алгоритма аллокации: алгоритм Мак-Кьюзи-Кэрелса и алгоритм двойников

Описание каждого из исследуемых алгоритмов

McCuzyAllocator Аллокатор

Мак-Кьюзи-Кэрелса (McCuzyAllocator) представляет собой пользовательский менеджер памяти, который работает, разделяя один большой пул на несколько блоков. При инициализации аллокатор резервирует крупный непрерывный участок памяти (например, через `malloc`) и описывает его специальным заголовком (Header), где хранится размер, связь с предыдущим блоком, флаг «свободно или занято» и т. д. При выделении (`allocate`) аллокатор ищет подходящий свободный блок — либо от начала (`first fit`), либо по другой стратегии. Найденный блок «отрезается» от свободного куска, при этом если

остаток достаточно велик, он остаётся новым свободным блоком. Освобождение (deallocate) помечает соответствующий блок как доступный и пытается «слить» его с соседними, если те тоже свободны. Это слияние (coalescing) уменьшает внешнюю фрагментацию: вместо нескольких маленьких свободных фрагментов образуется один крупный свободный блок. Таким образом, McCuzyAllocator может гибко работать практически с любыми размерами, создавая или объединяя блоки по мере надобности и фактически самоуправляя внутренней структурой пула. В некоторой реализации (как в вашем случае) величину запрашиваемого блока округляют до ближайшей степени двойки, упрощая расчёты и деление. Ключевые особенности: гибкое распределение памяти, возможность дефрагментации путём объединения соседних свободных блоков, а также потенциально более высокий «фактор использования» при «хорошем» распределении (особенно если размеры блоков сильно варьируются). Минусом может быть более затратный поиск подходящего блока и более сложная логика объединения, а также возможные накладные расходы на множество заголовков.

Аллокатор двойников

Аллокатор двойников (по степеням двойки), или buddy-аллокатор (в упрощённом виде — Allocator_pow_of_2), принципиально опирается на то, что память делится на блоки фиксированных размеров, каждый из которых — степень двойки (16, 32, 64, 128 байт и т.д.). При инициализации он либо разрезает большой пул на несколько списков, в каждом списке блоки одного размера, либо (в классической версии) рекурсивно делит блоки при потребности. В упрощённом варианте пользователь задаёт «сколько» штук 16-байтных блоков, 32-байтных, 64-байтных и т.д. нужно, и аллокатор сохраняет их как свободные в соответствующие «кучи». Когда программа запрашивает память размером N, аллокатор находит ближайшую степень двойки, не меньшую N, и берёт блок из списка этого размера (или возвращает nullptr, если блоки в этой «куче» закончились). Возврат памяти (deallocate) заключается лишь в том, чтобы поместить блок обратно в список свободных соответствующего размера. Такой подход даёт очень быструю работу (выделение и освобождение — это пара операций с указателями), но порождает «жёсткое» округление запросов (например, 20 байт превращается в 32, 65 в 128 и т.д.), что ведёт к росту внутренней фрагментации. Также, если заранее неверно оценить, сколько блоков какого размера может потребоваться, возможны

частые отказы при allocate («Zero free element»), даже если в целом памяти ещё достаточно, но она «раскидана» по другим размерам. Зато алгоритм двойников особенно хорош, когда размеры запросов предсказуемы и близки к степеням двойки.

Сравнение простоты использования

Аллокатор Мак-Кьюзи-Кэрелса, как правило, достаточно создать с указанием одного параметра (объём пула), а дальше он сам управляет различными размерами блоков и объёмом свободного пространства. Аллокатор по степеням двойки в базовом варианте потребует заранее определить, сколько блоков каждого размера нужно, что усложняет настройку, если распределение запросов заранее неизвестно. Однако при непосредственном вызове методов allocate/deallocate аллокатор « 2^n » работает крайне просто и быстро, так как вся логика сводится к выбору нужной «кучи» (списка) и манипуляциям с указателями. В итоге, если говорить о простоте начального использования и отсутствии лишних параметров, Мак-Кьюзи-Кэрелса может быть более удобен, а при акценте на скорости вызовов аллокатор по степеням двойки выглядит проще.

Код программы

mcquzy.hpp

```
#ifndef MCCUZY_H
#define MCCUZY_H

#include <cmath>

class McCuzyAllocator {
public:
    typedef void value_type;           // показал что аллокатор выдаёт безтиповую память
    typedef value_type *pointer;
    McCuzyAllocator() = default;
    ~McCuzyAllocator() { ::free(pointer_start); }
```

```

void free() {
    // static_cast - приведение типа к хедер
    auto *header = static_cast<Header *>(pointer_start); // указатель на начало пула памяти
помечаю его

                                //как указатель на стр. Header
    header->size = (size_total - size_header); // размер первого блока равен всему размеру
памяти
    header->available = true;          // пометил как доступный

    size_used = size_header; // сбросил счётчик
};

//округляю size вверх до ближайшей степени двойки
size_t approximation(size_t size) {
    int i = 0;
    while (pow(2, i) < size) {
        i++;
    }
    return (size_t)pow(2, i);
};

pointer allocate(size_t size) { // возвращаю указатель
    if (size <= 0) {
        std::cerr << "size must be bigger than 0\n";
        return nullptr;
    }

    size = approximation(size); // округлил запрошенный размер до ближайшей степени 2
    if (size > size_total - size_used) { // если размер двойки больше, чем памяти то,
возвращаем nullptr
        return nullptr;
    }
    //find()
    auto *header = find(size); // ищу блок памяти свободный
    if (header == nullptr) {
        return nullptr;
    }

    block_separation(header, size); //если блок больше нужного раздаю

    return header + 1;

};

void deallocate(pointer ptr) {

```

```

    if (!check_address(ptr)) {
        return;
    }

    auto *header = static_cast<Header *>(ptr) - 1;    // получил указатель на заголовок
освобождаемого блока

    header->available = true;
    size_used -= header->size; // уменьшил счётчик
    defragmentation(header); //пытаюсь слить этот блок с соседним

};

explicit McCuzyAllocator(size_t size) {
    size = approximation(size); // округляем запрошенный размер

    if ((pointer_start = malloc(size)) == nullptr) { // выделил память
        std::cerr << "failed to allocate memory\n";
        return;
    }

    size_total = size;

    pointer_end = static_cast<void *>(static_cast<char *>(pointer_start) + size_total);
    // привожу к чару для адресной арифметики
    // указывает на первый байт за концом выделенного пула

    auto *header = (Header *)pointer_start;
    header->size = (size_total - size_header);
    header->size_previous = 0; // предыдущего блока нет
    size_used = size_header;
    header->available = true; // начальный блок тут свободный

};

size_t getRealBlockSize(void* userPtr) {
    if (!userPtr) return 0;
    auto hdr = reinterpret_cast<Header*>(userPtr);
    hdr = hdr - 1;
    return hdr->size + sizeof(Header);
};

private:
    struct Header {
    public:

```

```

size_t size;
size_t size_previous;
bool available; //флаг доступности
inline Header *next() { return (Header *)((char *) (this + 1) + size); } //следующий блок
inline Header *previous() { //предыдущий блок
    return (Header *)((char *) this - size_previous) - 1;
}
};

const size_t size_header = sizeof(Header); // просто конст. размера структуры
pointer pointer_start = nullptr;
pointer pointer_end = nullptr;
size_t size_total = 0;
size_t size_used = 0;

Header *find(size_t size) {
    auto *header = static_cast<Header *>(pointer_start);

    while (!header->available || header->size < size) { //скипаю недоступные блооки и
        маленькие блоки
        header = header->next();
        if (header >= pointer_end) {
            return nullptr;
        }
    }
    return header;
    // вернул указатель на ptr
}

void block_separation(Header *header, size_t chunk) {
    size_t size_block = header->size;
    header->size = chunk; //отрезал кусок
    header->available = false; //занял блок !
    if (size_block - chunk >= size_header) { // проверка на остаток размера
        auto *next = header->next(); // указатель на некст блок
        next->size_previous = chunk; // размер предыдущего или только что занятого
        next->size = size_block - chunk - size_header;

        next->available = true;

        size_used += chunk + size_header;

        auto *followed = next->next(); //следующий х2 блок
        if (followed < pointer_end) {
            followed->size_previous = next->size; // если ок даю след блоку этот адрес

```

```

    }
} else {
    header->size = size_block; // если места нехватило то даю в сайз весь блок
    size_used += size_block;
}
}

bool check_address(void *ptr) {
    auto *header = static_cast<Header *>(pointer_start);
    // Начинаем с первого блока (заголовка) в пуле
    while (header < pointer_end) {
        if (header + 1 == ptr) {
            return true;
        }
        header = header->next();
        // Переходим к следующему блоку и повторяем
    }
    return false;
}

//объединяю блок с соседним
void defragmentation(Header *header) {
    if (previous_free(header)) {
        auto *previous = header->previous();
        if (header->next() < pointer_end) {
            header->next()->size_previous += previous->size + size_header;
        }
        previous->size += header->size + size_header; //увеличил размер предыдущего
        size_used -= size_header;
        header = previous;
    }
    if (next_free(header)) { //если следующий блок свободен
        header->size += size_header + header->next()->size; // сложил размеры
        size_used -= size_header;
        auto *next = header->next();
        if (next != pointer_end) {
            next->size_previous = header->size;
            //перезаписал size_previous
        }
    }
}

//существует ли предыдущий блок и доступен
bool previous_free(Header *header) {
    auto *previous = header->previous();

```



```
    return header != pointer_start && previous->available;

}

//существует ли следующий блок
bool next_free(Header *header) {
    auto *next = header->next();
    return header != pointer_end && next->available;

}
};
#endif
```

pow2.hpp

```
#pragma once

#include <cmath>
#include <iostream>
#include <unordered_map>    // для хранения списков свободных блоков
#include <vector>           // для удобного управления списком блоков

class Allocator_pow_of_2 {
public:
    struct Header { // содержит указатель next на следующий свободный блок
        Header *next;
    };

    std::unordered_map<size_t, Header *> free_blocks_lists; // списки свободных блоков
        // размер блока
        // указатель на связный список блоков

    std::vector<size_t> powsOf2 = {16, 32, 64, 128, 256, 512, 1024}; // возможные блоки

    void *start_point; // ук. на пул
    void *data;
    void *end_point; // ук. на кон. пула

    static size_t align(size_t size) { // привожу размер к ближайшему допустимому блоку 2^N
        int i = 0;
        while (pow(2, i) < size || pow(2, i) < 16) {
            i++;
        }
        return (size_t)pow(2, i);
    }

    Allocator_pow_of_2(std::vector<size_t> &blocks_amount);
    ~Allocator_pow_of_2();
    void *allocate(size_t bytes_amount);
    void deallocate(void *ptr);
};

Allocator_pow_of_2::Allocator_pow_of_2(std::vector<size_t> &blocks_amount) {
    // конструктор
    size_t bytes_sum = 0;
    for (size_t i = 0; i < blocks_amount.size(); i++) {
```

```

    bytes_sum += blocks_amount[i] * (sizeof(Header *) + powsOf2[i]);
    // кол-во блоков*(размер хедера + размер самого блока)
}

data = malloc(bytes_sum + (sizeof(Header *) * blocks_amount.size())); //выделяю пул
    // размером bytes_sum+ кол-во блоков * размер структуры
start_point = data;

end_point = (void *)((char *)start_point + bytes_sum + (sizeof(Header *) *
blocks_amount.size()));
// адрес конца пула начало + размер всего пула

void *point_now = start_point; // установил текущий поинтер

for (size_t i = 0; i < blocks_amount.size(); i++) {
    Header *head = (Header *)point_now;
    free_blocks_lists[powsOf2[i]] = head; //пустой связный список для каждого размера

    point_now = (void *)((char *)point_now + sizeof(Header *)); //сдвигаю point_now на размер
одного указателя

    for (size_t j = 0; j < blocks_amount[i]; j++) {
        head->next = (Header *)point_now; //связываем голову списка с следующим блоком

        head = head->next;

        point_now = (void *)((char *)point_now + powsOf2[i] + sizeof(Header *));
        //теперь указывает на позицию для следующего блока данного размера

    }
    head->next = nullptr;
    //последний блок null
}

// функция allocate
void *Allocator_pow_of_2::allocate(size_t bytes_amount) {
    if (bytes_amount == 0) {
        std::cerr << "Size must be bigger than 0\n";
        return nullptr;
    }

    size_t size = align(bytes_amount); //округлил до степени двойки
    Header *head = free_blocks_lists[size]; //адрес списка блоков типа хед
    if (head->next == nullptr) { // проверяю существование блоков
        std::cerr << "Zero free element\n";
    }
}

```

```

    return nullptr;
}
Header *alloc_elem = head->next; // взял первый блок
head->next = head->next->next; // заменяю первый блок на второй
alloc_elem->next = head;

return (void *)((char *)alloc_elem + sizeof(Header *)); // возвращаю указатель на начало
данных внутри выделенного блока
}

// Реализация функции deallocate
void Allocator_pow_of_2::deallocate(void *ptr) {
    if (ptr < start_point || ptr > end_point) {
        std::cerr << "Uncorrect ptr\n";
        return;
    }

    Header *ptr_header = (Header *)((char *)ptr - sizeof(Header *));

    for (size_t i = 0; i < free_blocks_lists.size(); i++) {
        if (free_blocks_lists[powsOf2[i]] == ptr_header->next) {
            ptr_header->next = free_blocks_lists[powsOf2[i]]->next;
            free_blocks_lists[powsOf2[i]]->next = ptr_header;
            return;
        }
    }
    std::cerr << "Uncorrect ptr\n";
}

Allocator_pow_of_2::~Allocator_pow_of_2() { free(data); } // деструктор

```

mcquzy.cpp

```

#include <ctime> // для clock()
#include <iostream>
#include <vector>
#include "../include/mcquzy.hpp" // ваш заголовок

using namespace std;

int main() {
    McCuzyAllocator allocator1(1024 * 1024); //аллокатор на 1мб
    clock_t start, end;

```

```

vector<void*> blocks(1000);

start = clock();
for (int i = 0; i < 1000; i++) {

    blocks[i] = allocator1.allocate(i + 1); // блоки размером i+1
}
end = clock();
cout << "McCuzyAllocator:"<< endl;
cout << "Общее время выделения: " << (double)(end - start)/(double)CLOCKS_PER_SEC
<< " sec. "<<endl;

start = clock();
for (int i = 0; i < 1000; i++) {
    allocator1.deallocate(blocks[i]);
}
end = clock();
cout << "Общее время освобождения: " << (double)(end -
start)/(double)CLOCKS_PER_SEC << " sec. " << endl;

size_t sumRequested = 0; // всего байтов
size_t sumAllocated = 0; // реально занято

for (int i = 0; i < 1000; i++) {
    size_t req = i + 1; // запрос

    void* p = allocator1.allocate(req);
    if (!p) {
        break;
    }
    blocks[i] = p;
    sumRequested += req; // байтов выделил
    size_t real_size = allocator1.getRealBlockSize(p); // беру реальный размер аллокатора
    sumAllocated += real_size;
}

// вычисляю фактор использования
double factor = 0.0;
if (sumAllocated > 0) {
    factor = (double)sumRequested / (double)sumAllocated;
}
cout << "Фактор использования: " << factor << endl;

// высвободил
for (int i = 0; i < 1000; i++) {
    if (blocks[i]) {

```

```

        allocator1.deallocate(blocks[i]);
    }
}

return 0;
}

```

pow2.cpp

```

#include <ctime>
#include <iostream>
#include <vector>
#include "../include/pow2.hpp"
using namespace std;

int main() {
    vector<size_t> k = {16, 16, 32, 64, 128, 256, 512};

    Allocator_pow_of_2 alloc(k);
    clock_t start, end;
    vector<void*> blocks(1000);

    start = clock();
    for (int i = 0; i < 1000; i++) {
        blocks[i] = alloc.allocate(i + 1); // округляется, берётся из списка
    }
    end = clock();
    cout << "Allocator 2^n:" << endl;
    cout << "Общее время выделения: " << (double)(end - start)/(double)CLOCKS_PER_SEC
    << " sec. " << endl;

    start = clock();
    for (int i = 0; i < 1000; i++) {
        alloc.deallocate(blocks[i]);
    }
    end = clock();
    cout << "Общее время освобождения: " << (double)(end -
start)/(double)CLOCKS_PER_SEC << " sec. " << endl;

    //фактор использования
    size_t sumRequested = 0;
    size_t sumAllocated = 0;

```

```

for (int i = 0; i < 1000; i++) {
    size_t req = i + 1;
    void* p = alloc.allocate(req);
    if (!p) {
        break;
    }
    blocks[i] = p;

    sumRequested += req;

    size_t aligned = Allocator_pow_of_2::align(req); // размер к ближайшему допустимому
    блоку 2^N
    size_t overhead = sizeof(Allocator_pow_of_2::Header); // размер заголовка

    sumAllocated += aligned + overhead;
}

double factor = 0.0;
if (sumAllocated > 0) {
    factor = double(sumRequested) / double(sumAllocated);
}
cout << "Фактор использования: " << factor << endl;

// освобождаем
for (int i = 0; i < 1000; i++) {
    if (blocks[i]) {
        alloc.deallocate(blocks[i]);
    }
}

return 0;
}

```

Протокол работы программы

user@DESKTOP-Q8EOOMG:/mnt/c/Users/user/Desktop/Labs/os/cp/build\$./pow2

Allocator 2^n:

Общее время выделения: 0.000227 sec.

Общее время освобождения: 0.000443 sec.

Фактор использования: 0.733269

user@DESKTOP-Q8EOOMG:/mnt/c/Users/user/Desktop/Labs/os/cp/build\$./mcquzy

McCuzyAllocator:

Общее время выделения: 0.002882 sec.

Общее время освобождения: 3.1e-05 sec.

Фактор использования: 0.716561

user@DESKTOP-Q8EOOMG:/mnt/c/Users/user/Desktop/Labs/os/cp/build\$

Вывод

В работе реализованы и проанализированы два аллокатора памяти: Мак-Кьюзи-Кэрелса и по степеням двойки. Аллокатор Мак-Кьюзи-Кэрелса даёт гибкое выделение блоков и умеет эффективно объединять свободные участки, но поиск подходящего блока может быть более затратным. Аллокатор по степеням двойки упрощает и ускоряет операции за счёт жёсткого набора размеров, однако при «неудачных» запросах существенно растёт внутренняя фрагментация. По результатам измерений, « 2^n »-аллокатор работает быстрее при выделении и освобождении, тогда как Мак-Кьюзи-Кэрелса более экономно расходует память. Таким образом, выбор аллокатора зависит от приоритетов: если важна максимальная скорость, подходит « 2^n »-аллокатор, а при требовании более рационального использования памяти лучше Мак-Кьюзи-Кэрелса.